

e overall heading of the page. If you think of content like a hierarchy of importance, text should be a short and sweet summary of the content that the visitor will read. It is best practice to add a value here, as it has a large negative impact on SEO if it is missing (More on H1 tags here).

Top navigation configuration

The Navigation CTA Text and Navigation CTA Link are used to add a link to the top navigation. The top navigation consists of the GitLab logo, the Partner logo (if added) and this call-to-action button. If the Navigation CTA text is not entered, the button will not render.

Note: For links to an email address, you must add `mailto:` before the email address value. Ex: `mailto:myemail@test.com`. This applies to any other button on the page. In Markdown, that would look like `Contact us`.

Visual Representation

Side menu configuration

The side navigation block allows you to add two content blocks: Tab Controls Container and Side Navigation. These content types are used throughout our marketing site. Because of that, these blocks are created to work in differing page types and may contain fields that are not relevant to the microsite pages.

Contentful will allow you to add more than one of each, but this may result in the page breaking. Please only add one of each (the tab controls is optional). You can also re-use existing blocks that you may have built for other microsite pages.

The following tables contain fields necessary to build out the side navigation blocks. It leaves out fields that are not used. If you enter values in the un-used fields, you will not see any changes related to them.

Tabs configuration

Tab Controls Container	Description
Example Value	
-----	-----
-----	-----
Internal name	This field does not appear on the page, but is used for internal purposes such as search.
Header	Appears above the tabs
"Try Ultimate for free"	
Tabs	These are the individual tab blocks that build up the tab controls container. They have their own set of fields.
This takes in the Tab Control block	

Tab Controls	Description
Example Value	
-----	-----

```

----- |
----- |
| Internal name | This field does not appear on the page, but is used for internal purposes
such as search. | "SaaS tab"
|
| Tab ID | Required for the tabs to work correctly and keep them accessible. Must be
unique and lowercase. You can keep this value simple. | "saas"
|
| Tab Button Text | The text that appears on the tab. Because of room constraints, try to
keep this value short. | "GitLab.com"
|
| Tab CTA | This will take in a Button content type. This is optional.
| This takes in the Button block |
| Tab Panel Content | This will present you with multiple options, but at this time, you can
only use the Text block. Nothing else will work for these pages. | Use the Text block here
|

```

```

| Button | Description
| Example Value |
| ----- | -----
-----
----- |
----- |

```

```

----- |
| Internal name | This field does not appear on the page, but is used for internal purposes
such as search.
| "Microsite - SaaS Trial CTA" |
| Text | Button text
| "GitLab.com trial" |
| External URL | The url that you want to take page visitors to. Remember that the
utmpartnerid will be appended automatically, so no need to add that here! You may have other
utm parameters you wish to add here, though.
| "https://gitlab.com/-/trialregistrations/new" |
| Data GA name | This is used for Google Analytics. You may enter a value here (the best
practice is the name of the button text, but in all lowercase). However, if a value is not
entered, there is logic behind-the-scenes to create these values in Google Analytics
automatically! | "gitlab.com trial" |
| Data GA location | This is used for Google Analytics. You may enter a value here (the best
practice is the location of the button, but in all lowercase). However, if a value is not
entered, there is logic behind-the-scenes to create these values in Google Analytics
automatically! | "side menu - saas tab" |

```

```

| Text | Description |
| Example Value |
| ---- | -----
-----

```

```

| Text | This is in Markdown. You can use Markdown syntax or the field's text formatter here. |
"We host. Get started right away - no technical setup or installation required." |

```

Side navigation configuration

Side navigation	Description
Example Value	
-----	-----
-----	-----
Internal name	This field does not appear on the page, but is used for internal purposes such as search.
"Microsite - side nav"	
Anchors	This takes in Anchor Link blocks. The ones entered here will be at the first level of nesting in the side navigation. Use the Anchor Link block here

Anchor link	Description
Example Value	
-----	-----
-----	-----
-----	-----
-----	-----
Internal name	This field does not appear on the page, but is used for internal purposes such as search.
"Microsite - DevSecOps"	
Link Text	The text used for the link.
"DevSecOps"	
Anchor Link	No external link should be added here. The anchor link MUST include a before the text, should be lowercase, and all words must be separated by -. This should match up with an Anchor ID value you enter in the track content. Please make sure you test these links before publishing the page. "devsecops"
Child Anchor Links	You can add a new 'Anchor Link' content entry here. Please note that our side navigation component only renders one level of child links. Use the Anchor Link block here
Nodes (DEPRECATED)	This is an older, more complicated way of adding child links. See the new Child Anchor Links field. These are optional, but allow for a second level of nesting in the side menu. (Ex: if you want to link to sections within the DevSecOps track). At this time, this only takes in JSON data. We are working to make this easier to use. See below for code snippet. (Consider copy/pasting this and editing to your needs)
Data GA name	This is used for Google Analytics. You may enter a value here (the best practice is the name of the button text, but in all lowercase). However, if a value is not entered, there is logic behind-the-scenes to create these values in Google Analytics automatically! "what-is-gitlab"
Data GA location	This is used for Google Analytics. You may enter a value here (the best practice is the location of the button, but in all lowercase). However, if a value is not entered, there is logic behind-the-scenes to create these values in Google Analytics automatically! "side menu"

<details><summary>Expand to view Nodes code snippet</summary>

Note: This method is being deprecated. Please use the Child Anchor Links field instead!

JSON

```
{
  "nodes": [
    {
      "href": "what-is-gitlab",
      "text": "What is GitLab?"
    },
    {
      "href": "your-link-here",
      "text": "Your Text Here"
    }
  ]
}
```

</details>

Visual Representation

<details><summary>Expand to view screenshot</summary>

</details>

Footer CTA configuration

The Footer CTA looks similar to the footer CTAs across our site (also known as "Next Steps"). The partner microsite template uses a page-specific footer CTA content type, · Partner Microsite - Footer CTA because this design slightly strays away from the other variants across our site.

If you have entered an image in the Microsite entry's Partner Logo field, it will also show up in this section. You will not see this within the Footer CTA entry in the CMS.

Footer CTA	Description
Example Value	
-----	-----
-----	-----

Internal name	This field does not appear on the page, but is used for internal purposes such as search.
	"Microsite - Basic Footer CTA"
Heading	The main, large text of the section
"Ready to get started?"	
Tagline	The subtext shown under the heading.
"See what your team could do with GitLab, the most comprehensive AI-powered DevSecOps platform."	
Left Column CTA text (optional)	The footer section is divided into two columns. This line

acts as a header for the left column. | "Try Ultimate for free"

|

| Left Column Buttons (optional) | You can leave this empty or have up to two buttons.

| Add a Button entry here

|

| Right Column Taglines (optional) | You can leave this empty or have up to two taglines here.

This takes in a Header and Text content entry | Add a Header and Text content entry here

|

Visual Representation

<details><summary>Expand to view screenshot</summary>

</details>

Microsite track/section configuration

The · Partner Microsite takes in individual sections, or tracks, containing content. These blocks are a content type called · Partner Microsite - Track. By splitting the page up into these tracks you can easily reuse them across different partner pages. In addition, the page's logic knows to add some spacing in between these sections to visually break them up.

The content type takes in an Internal ID and a series of available content blocks. The following content blocks are available within a Track at this time: Header, Subheader, and Text, External Video, Card Group, and Card.

Header, Subheader, and Text

You can use this content type's fields interchangeably, meaning you don't always need to use the header or text. This is useful for when you want to create a large block of text, then add a video or card group in the middle, and want to continue with text just below it.

| Header, Subheader, and Text | Description

| Example Value |

| ----- | -----

--- | ----- |

| Internal name | This field does not appear on the page, but is used for internal purposes such as search.

| "Microsite - Built-in Security" |

| Anchor ID (optional) | This can be used in tandem with the side navigation links. The value should NOT contain a symbol. The value should match with the side navigation link value.

| "security-and-compliance" |

| Header (optional) | If you would like to use a header, the value would go here.

| "With Built-in Security, Move Forward, Faster" |

| Subheader (optional) | If you would like to use a subheader, the value would go here.

| "Protect the integrity of Your Software Supply Chain" |

| Text (optional) | If you would like to add text, you can add long blocks of text

here. You can use this content block for just a block of text with no headers.
| Long-form, formatted text can go here. |

Visual Representation

<details><summary>Expand to view screenshot</summary>

</details>

<details><summary>Example of continued text</summary>

</details>

External Video

It is best practice for videos to be uploaded to Vimeo, however our Video content type allows Youtube video URLs. If you need a video uploaded to one of these platforms, please reach out to the Brand/Video team. We do not directly upload videos to the CMS.

You can find the embed URL of the video by going to that platform's Share button, look for something that mentions Embed, and copy the URL (not the whole chunk of code) that it gives you.

External Video	Description
----------------	-------------

Example Value	
---------------	--

--	--

-----	-----
-------	-------

-------	--

-------	--

-------	--

External video	Title of the video, used for internal search
----------------	--

"What is GitLab?"	
-------------------	--

--	--

Title	Used in the iFrame that the video renders in. Useful for accessibility purposes.
-------	--

"What is GitLab?"	
-------------------	--

--	--

URL	The embed URL of the Youtube or Vimeo video.
-----	--

"https://player.vimeo.com/video/799236905?h=4eee39a447&badge=0&autoplay=0&autoplayid=0&appid=58479"	
---	--

Anchor ID (optional)	To be used in tandem with the side navigation. Some videos could be used as standalone content in their track. This allows a page visitor to easily scroll to them.
----------------------	---

"what-is-gitlab"	
------------------	--

--	--

Visual Representation

<details><summary>Expand to view screenshot</summary>

</details>

Card Group

The card group should be used when two or more cards should be blocked together. There are two different types of configurations that will work at this time.

You will configure the card group as normal, but the way you configure the individual Card will differ.

Card Group	Description
Example Value	

Internal name	This field does not appear on the page, but is used for internal purposes such as search.
Cards	This will reference Card entries.
Add existing, or create new Card entries	

Header and text variation

Card Group -> Card - Header and Text	Description
Example Value	

Internal name	This field does not appear on the page, but is used for internal purposes such as search.
Title	Main heading of card
Description	Text block within the card
Use Markdown / the text editor to add a formatted text block here	

Visual representation

<details><summary>Expand to view screenshot</summary>

</details>

Icon and header variation

If no text is entered, there is code in place to make the heading font a little bigger in order

to look nicer. This is paired with an icon of your choice.

At this time, our icons render depending on the icon name that you enter in the field. Our icon library lives in our design system, Slippers. When you access this link, there is a dropdown called Icon on the left. This contains our current icon library. You can view the icon once you select it from the dropdown. When you find an icon you want to use, type in that name \minus the "slp-" in the "Icon" field in the CMS. Ex: "slp-digital-transformation" should be "digital-transformation".

We are actively working on a solution to make the icon selection experience much easier!

Card Group -> Card - Icon and header Description	
Example Value	
-----	-----
-----	-----
Internal name	This field does not appear on the page, but is used for internal purposes such as search.
DevSecOps"	"Microsite - Best-in-class
Title	Main heading of card
"Best-in-class DevSecOps"	
Icon Name	See notes above. This should be an icon found in Slippers. Do not enter the "slp-" portion of the icon name.
	"speed-alt-2"

Visual representation

<details><summary>Expand to view screenshot</summary>

</details>

Card

The card is the same that is used within the Card Group, except the visual representation is different. A Card used outside of a Card Group will be a standalone section of content with a colored background. It can be helpful to think of this content block to "showcase" an important list.

At this time, these call-out sections take in a list of information. The checkmark icon is automatically pre-pended for each list item.

Card	Description
Example Value	

-----	---

| Internal name | This field does not appear on the page, but is used for internal purposes such as search. | "Microsite - automated software delivery card"

|

| List | For each list item, type in the sentence and press ENTER

| ["Integrate security across the entire DevOps lifecycle", "Improve collaboration across development, security and operations teams", "Speed up their cloud-native, hybrid, and multi-cloud adoption"]

Visual representation

<details><summary>Expand to view screenshot</summary>

</details>

Useful editing tips and tricks

1. Contentful's key feature is that it allows you to reuse content. Create once, use everywhere! You can look on the right side of an entry under Links to see where the entry is being used. If you make an update to a shared entry, it will update everywhere. This is good or bad depending on your intentions! If you want to make an update but not have it be reflected everywhere, it may mean that a new entry of some type is warranted.
2. We have functionality on these partner pages that auto-appends the partner ID to links. This does NOT work for links created in Markdown. You will have to add these in yourself to ensure proper Google Analytics tracking. That format looks like link name{data-ga-name="your value in lowercase goes here" data-ga-location="body"}
3. Some resources, like videos, may already be added to Contentful. Before adding new content, search around and see if you can reuse something already available.
4. It is better practice for a track to start off with the Header, Subheader, and Text or External Video content block, as they can contain an Anchor ID. A Card Group or standalone Card will visually fit better between text blocks or directly below a text block.

Video demo and overview

<figure class="videocontainer">

<iframe src="https://www.youtube.com/embed/kALB1thn6ac?si=3GV9LK4v2-L5y9XP" frameborder="0" allowfullscreen="true"> </iframe>

</figure>

SECTION: marketing/digital-experience/contentful-cms/solutions-pages.md

Solutions Pages

The Digital Experience team has selected the solutions pages (/solutions/) to be managed and served within Contentful. This page is designed to allow users to create a new solutions page from scratch using Contentful as CMS.

For in-depth information about Contentful CMS, please visit our Resource Deep Dive page.

How to make a solutions industry page from scratch

Basic structure

For now we only have an industry template to choose from in order to create solutions pages. More options will be added to this documentation in the future.

Setting up your URL in the slug field

A solutions template should be an entry of content type Page and also have solutions somewhere in its URL as a slug for it to be built with Solutions template. A common approach is to name your page slug as solutions/<your-page-name-here>. However, you could also have 2 level nested solutions pages that would have a slug composed like solutions/<your-page-here>/<child-page-here>.

Note that there is not a leading nor a trailing slash in the URL. Thi is important to take into account for creating solutions pages.

Metadata Configuration

To configure SEO metadata attributes of a solutions page, it's necessary to create a reference entry using the SEO Content Type.

Then, you can link the SEO configuration back in your page, as shown below.

Industry template declaration

Furthermore, you also need to declare your solutions page as an industry template. To do so, add the following JSON attributes in the schema field of your page for setting it up as an industry template.

```
json
{
  "template": "industry"
}
```

A basic solutions page with the industry template is composed of a Hero, a list of components

and a NexStep component (more on this later) at the bottom of the page. You can rearrange the components to suit a specific order within Contentful, as long as they are named properly. You can only choose from a set of supported components for this template.

A component has to be named within the industry template for it to be recognized and compile correctly. This means that a component with no name, an error in its name or if it's component not recognized by the template.

Some existing pages that follow the solutions industry template are:

- /solutions/agile-delivery/
- /solutions/value-stream-management/
- /solutions/continuous-integration/
- /solutions/security-compliance/
- /solutions/delivery-automation/
- /solutions/public-sector/
- /solutions/startups/
- /solutions/startups/join/
- /solutions/open-source/
- /solutions/open-source/join/
- /solutions/open-source/partners/
- /solutions/nonprofit/
- /solutions/nonprofit/join/

Available components

Components that can be used as of today within the solutions industry template.

Most used components:

- SolutionsHero
- BySolutionValueProp
- BySolutionIntro
- ByIndustryIntro
- ByIndustryQuotesCarousel
- Copy
- CopyMedia
- SolutionsResourceCards
- TierBlock
- Faq
- QuotesCarousel

Can still be used as components:

- AnalyticsInsightsCustomerSuccessCard
- AnalyticsInsightsFeatured
- AnalyticsInsightsShowcaseFaq
- GroupButtons
- NewQuotesCarousel
- NextStep

- NextStepAlt
- PricingTiers
- ReportCta
- SlpBreadcrumb
- BySolutionAccordeon
- BySolutionBenefits
- BySolutionCasestudies
- BySolutionHero
- BySolutionLink
- BySolutionListImage
- BySolutionShowcase
- BySolutionValueProp
- ByIndustryCaseStudies
- ByIndustrySolutionsBlock
- OpenSourceBlog
- OpenSourceFormSection
- OpenSourcePartnersGrid
- OverviewBlocks
- PartnersGridFull
- CtaBlock
- ResourceCarousel
- SolutionsHero
- SolutionsVideoFeature
- CustomerLogos
- EducationCaseStudyCarousel
- StartupsIntro
- StartupsLink
- StartupsOverview
- NonprofitIntro
- NonprofitOverview
- SingleCtaBlock

How to include a Sidenav

A SideNavigation is declared like any other component at the root pageContent field of the Page you are building.

How to include components within a Sidenav

SideNavigations are configured within contentful in a way that they can hold components nested within it (components that have links).

We can see in the image that the components are nested inside the SideNavigation instead of the root content of your page.

NextStep component

You can have a classic NextStep component at the bottom of your page by default, which will always show up at the bottom of the page.

Otherwise, you could include a NextStepAlt attribute to set this variant of the NextStep component. Just as before when we declared our page to follow the industry template, we can declare it to use the alternative version of the NextSteps component. You would then have a schema field in your page that looks similar to this:

```
json
{
  "template": "industry",
  "nextstepalt": true
}
```

SECTION: marketing/digital-experience/contentful-cms/the-source.md

"The Source" Pages

The Digital Experience team has selected the solutions pages (/the-source/) to be managed and served within Contentful. This page is designed to allow users to create a new "The Source" page from scratch using Contentful as CMS.

For in-depth information about Contentful CMS, please visit our [Resource Deep Dive](#) page.

How to Edit/Create pages using "The Source" template

For our Buyer Experience Site, creating or updating a The Source page requires the use of the [The Source Content Type](#).

Each entry in this Content Type functions as a distinct page on the site, complete with its Metadata configuration, Routing configuration, Hero configuration and Content Body with markdown.

Here is direct link to view all of our source article entries in Contentful.

Click on the article you'd like to edit from that list. You can use also click the "add entry" blue bottom on the top left to create a new article. Select "The Source" content type.

Routing configuration

To configure the page's routing, it is essential to declare the slug field. The page routes can extend up to three levels being the first level the-source, for example, the-source/level-2/level-3.

The field won't accept routes with a trailing slash or a leading slash.

Metadata Configuration

To configure SEO metadata attributes of a [The Source](#) page, it's necessary to use the SEO Title and SEO Description fields of the content type

To improve search engine results, use different language for the slug, Headline, SEO Title, and sSEO Description.

URLs should be kept short and aim to include topical terms and no stop words. URLs that are concise, descriptive, and easy to understand are generally better for both SEO and user experience.

Title tag (SEO Title field) and H1 (Headline field) need to be interchangeable - a title tag is shown in SERPs and has to have a maximum length of 55-60 characters. H1s can be longer to be descriptive, also for optimisation purposes title tags need to be changeable/ similar for H1s, but independently.

The meta description (SEO Description feid) in SERPs is what entices action from a searcher, and should use as much of the 155-160 character limit as possible.

Hero Configuration

To personalize the hero section, use the fields in the · The Source Content Type. These fields are designed to let you customize the static Hero section, making it the first component that visitors see on the page.

"The Source" Page Field	Description	Example Value	Localizable
Headline	Main header of the Hero	"The Source Headline"	Yes
Date	Date of the article	"07/31/2024"	No
Time to read	Time to read the article	"7 min read"	Yes
Hero Image	Main hero image	Example image	No
Author Name	Name of the author	"Jane Doe"	No
Author Headshot	Photography of the author	Example image	No

Visual representation

Key Takeaways section

This section is located aside from the main content of the article, use the Key Takewaways field in the · The Source Content Type.

Visual representation

Body Content Section

This is the main article's content section, the field is called Article Body, this section can be edited using markdown and it has some features that can be used to render cards or buttons within markdown.

Buttons and Cards

To render buttons and cards using markdown, The Source allows to do it following the next markdown formatting:

Button:

```
markdown
link{class=button}
```

Card:

```
markdown
```

Card Content

Visual Representation

Example markdown:

```
markdown
You may also be interested in
```

```
>
> Main Header
>
> Lorem ipsum dolor est
> See how > {class="button"}
```

SECTION: marketing/digital-experience/contentful-cms/wishlist.md

Known Knowns

This are things we know aren't great about our Contentful integration with the Marketing site. They are "gotchas". We are documenting them here for transparency.

1. Just for reference, the slug field in the PAGE Content Type should not be marked as a localization field, if this field is checked it will break all localized pages in BE where the content comes from Contentful. (edited)
2. Diff Versioning in Contentful isn't great. Infact... it's non-existent.
3. Error handling: Unless a user is checking the most recent pipeline after publishing something in contentful, they may not be aware of a problem with the content. Users may subscribe to the dex-alerts slack channel to be notified when alerts occur, but the name of the alerts will be the name of the most recently merged branch in Buyer Experience, rather than the page that was edited in the CMS.
4. Draft content has the potential to break pipelines for all review apps, since review apps read contentful draft content.

Wishlist

This are a wishlist of features we'd like to see from Contentful or things we build.

1. To solve all the problems of the universe.

SECTION: marketing/digital-experience/decap-cms/_index.md

Where can I find blog posts?

Blog posts can be found in two different locations:

1. Yaml files in the about.gitlab.com Repository
2. Decap CMS

Creating a blog post in the about.gitlab.com repository

Creating a blog post from scratch is as easy as filling out the blog post template. This can be done using the WebIDE, or by setting up the project locally.

1. Copy the example yaml document, and place it under the /blog/ folder within the appropriate locale. For example, English blog posts would be under /content/en-us/blog/
2. If you're adding a new image, add it to the /public/images/blog/hero-images/ folder and update the file path in your yaml file.
3. Add an author (first name, last name). If the author does not already exist in this list of authors, you can create a new one by copying and pasting the format of an existing author.
4. Fill in the remaining sections of the yaml file, using markdown for the body of the blog post, and assign your merge request to @Sgittlen for review.

Creating a blog post using Decap CMS

mermaid

flowchart TD

A[Login into Decap with GitLab SSO] --> B{Are you creating a new article or editing an existing one?}

B -- "New Article" --> D[Click New Blogpost]

B -- "Editing Article" --> J{Published or Unpublished?}

J -- "Published" --> C[Click Contents Tab and Edit Existing Blogpost]

J -- "Unpublished" --> E[Go to Workflow Tab, find unpublished article, and click into it]

C --> F[Save Changes]

D --> F[Save Changes]

E --> F[Save Changes]

F --> G[MR is created with draft changes in GitLab. Review App is created based on the MR]

G --> H{Approval Decision}

H -- "Approve Changes" --> I[Merge changes into codebase]

H -- "Request Feedback / Update MR" --> F

H -- "Delete Article" --> K[Delete article: close MR and preserve contents]

1. Log into Decap by visiting <https://about.gitlab.com/admin/> and entering your GitLab credentials.
1. Using the language of your choice, select a Blog - Post Collection.
1. Fill in the fields under Content. Most fields are required unless otherwise noted. (You may also fill out the SEO fields - if you leave them blank, they will populate automatically with the Title and Description from the Content section. The last section, Config, can optionally be used to change the slug, and set the blog post to be Featured on the blog landing page.)
1. Click Save in the upper right corner.
1. Check the latest Merge Requests in the about.gitlab.com repository. A new merge request titled "Create Blog - Post {title}" will have automatically been created.
1. Within that MR, you will be able to view the review app of your blog post, and tag @Sgittlen for approval and merge.
1. Deleting a non-published post in Decap will end up closing the MR. If you are looking for that content, it will remain in the GitLab UI.

Video Tutorials can be found at the following playlist. For assistance, please reach out to digital-experience-team or blog in slack.

Additonal Notes

- Note that the swim lames/board in the Workflow tab correspond to labels applied to the various MRs.
- If you want to add an author, category, or tag, you will need to create those in Decap and merge the MRs in to the main branch first. Once that is complete the author, category, or tag will become available to be selected in Decap when creating or editing an existing article.

Adding media to blog posts

Mermaid charts

plaintext

mermaid

```
flowchart TB;
```

```
subgraph legend[ Legend ]
```

```
  proc(Process);
```

```
  art[Artifact];
```

```
  proc -->|IO relation|art;
```

```
end
```

Videos

html

```
<figure class="videocontainer">
```

```
<iframe width="560" height="315"
```

```
src="https://www.youtube.com/embed/pA5SfHwlq0s" frameborder="0"
```

```
allowfullscreen="true">
```

```
</iframe>
```

```
</figure>
```

Code blocks with syntax highlighting

plaintext

json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ecr:GetAuthorizationToken",
        "ecr:BatchCheckLayerAvailability",
        "ecr:GetDownloadUrlForLayer",
        "ecr:DescribeRepositories",
        "ecr:ListImages",
        "ecr:DescribeImages",
        "ecr:BatchGetImage"
      ],
      "Resource": ""
    }
  ]
}
```

Creating a blog post using the Web IDE

1. Visit the project repository at <https://gitlab.com/gitlab-com/marketing/digital-experience/about-gitlab-com>
1. Click the Code button dropdown, and select Open with Web IDE
1. Once open, navigate to the folder you'd like your blog post in
 - For most, this will be in content -> en-us -> blog
 - Localized blog posts can be found in the content/{{language-code}} folder, such as content -> fr-fr -> blog
1. Add a new file by clicking the Add file button at the top of the sidebar pane. Be sure to name your file whatever the url of the blog post will be
 - Alternatively, if you have an existing yaml file (i.e. from Argo), you can drag and drop the file directly into the folder of your choosing. Be sure to right click -> rename the file to match the English version
1. From here, you can copy and paste an existing blog post yaml file and fill in your new values, or use the blog MR template to start from scratch
 - Be sure to either upload your blog image inside /public/images/blog/hero-images/, or copy a URL from Cloudinary

1. Once you're happy with your post, you can create a merge request through the Web IDE by selecting the Source Control icon on the left sidebar, then click the dropdown beside the Commit and push to main button -> Create a new branch and commit (you won't be able to commit and push to main directly)
 - Hit Enter to just use the default branch name
 - Click the Create MR button that appears in the bottom corner
1. Wait for the pipeline to run to view your review app

For a video walkthrough, click [here](#).

SECTION: marketing/digital-experience/engineering/accessibility.md

Surface accessibility issues on about.gitlab.com

Open an issue to the Buyer Experience repository and add the dex-category::accessibility label.

Resolving Accesibility issues

Quick and dirty (Start here)

- > I have run Axe Devtools
- >
- > I have run Google Lighthouse

Notes about tabbing across browsers

Different browsers respond differently to tabbing through interactive elements (like links, buttons, form inputs, etc.) on a page. Chrome and chromium based browsers will have tab move to the next focusable element and shift + tab will have move back a focusable element. This is done to assist folks with screen readers.

Safari doesn't use tab navigation by default. The user has to go into Safari's setting and check the box to turn on tabbed navigation. Mozilla is being true to operating system settings in Mac OS, which means you also need to turn it on.

Related resources:

<https://stackoverflow.com/questions/52783375/anchor-with-href-not-taking-focus-in-safari-even-with-all-accessibility-keyboard>

<https://stackoverflow.com/questions/11704828/how-to-allow-keyboard-focus-of-links-in-firefox>

Guidance to tackling accessibility challenges related to tabbing

Properly set tabindex: You can use the tabindex attribute to specify the order in which elements receive focus when the user uses the tab key.

Properly use ARIA roles: ARIA roles provide a way to make web content is accessible to people with disabilities. They can be used to define the role of an element, helping assistive technologies understand how to interact with it.

Consistent design: Ensure that your design visually indicates which element currently has keyboard focus (ex: by changing the color or outline). This visual cue will help all users, not just those using assistive technologies.

More comprehensive testing

Tools like Axe and Google Lighthouse only take you so far, and are a good starting point for us. Using screen reader technology like VoiceOver on MacOS to test on a real webpage is needed to reach AA compliance.

> Run through this Accessibility spreadsheet:

<https://docs.google.com/spreadsheets/d/1pCfGv8xHjY39Oeft37pGjgotlg0j3cSyty0H9njv8/editgid=0>

Example of testing accessibility on our home page: <https://www.youtube.com/watch?v=jtVMFUQLJEE>

SECTION: marketing/digital-experience/engineering/buyer-experience-repository.md

Overview

Technical Architecture

1. Nuxt.js: A vue.js static site generator
2. Slippers Design System: SSoT for the marketing site's design system
3. Focus Resources: Devoted to two repositories
 - Digital Experience focused resources on the Buyer Experience and Slippers repositories.

Scope

1. Move 80 marketing webpage to the Buyer Experience repository.
 1. Pricing
 1. Homepage
 1. Free Trial
 1. Sales
 1. Demo
 1. Install
 1. Features (Landing and single)
 1. Topics (Landing and single)
 1. Solutions (Landing and single)
 1. Enterprise
 1. SMB
 1. Partners (Landing and single)
 1. Comparison Pages

See the Marketing Site Information Architecture Refresh epic for single source of truth of pages being migrated to this repository.

Requesting Help when the Information Architecture Refresh Impacts your Work

Other keywords that may help folks find this section when searching the handbook:

- Missing content on the marketing website
- My changes in the website get overwritten
- Can't find my files anymore
- Files were moved
- Information architecture migrated my content

In order to meet the goals of the information architecture refresh and serve the business, the Digital Experience team will need to move existing content, and in some cases, remove existing content.

We will do our best to find a solution that meets the business goals of the migration, and keeps your existing processes intact. We apologize in advance for any inconveniences incurred by this work. We will be doing our best to communicate with stakeholders in advance, but due to the scope of the migration, we may miss appropriate channels from time to time.

If we have moved or removed your content and it's causing problems for you, your team, customers, or other stakeholders, here are steps you can take to escalate these issues to us:

1. First, check the Buyer Experience repository and see if you can find corresponding YAML files for your content. The Web IDE editing experience will be the same as it was in the [www-gitlab-com](https://www.gitlab.com) repository.

1. If you are unable to find or edit your content, or have additional concerns, please start by opening an issue in the Buyer Experience issues board.

1. For urgent requests, you can find our team in the digital-experience-team Slack channel.

Content Editing Experience

Where Content Exists

All content is stored in the `/content` directory and can be accessed via the Nuxt content module

How to Edit Content

1. Edit content directly in the browser using GitLab Web Editor.

1. Alternatively, set up a local development environment and use the Nuxt content editing tools on your machine. Then commit your changes and make a merge request in GitLab.

How to Add a New Webpage

1. Check if the new page is part of our Site Architecture Plan. These are the only pages that will exist in the Buyer Experience repository. For all other pages, please create an issue with the Digital Experience team to add your page to the old static site.

1. If your page is part of an existing neighbourhood (i.e. `/solutions/`, `/partners/`, `/topics/`) the components will already be built for you. If it is a completely new page, please request support from the Digital Experience Team via this issue template.

1. Add the content of your page into the `/content` folder in a `.yaml` file. Please follow existing indentation and convention.

1. Create an MR, and tag both an Engineer and a Designer on the Digital Experience Team as a

Reviewer. Only the Digital Experience Team will be able to merge.

Will there be a content management system?

Probably not. As we reduce the size and scope of the marketing site, we will be raising the bar for the quality of our page design. Content management systems present significant challenges in quality control.

In order to keep the Buyer Experience looking as good as possible, we have decided not to allow for markdown or rich text editing experiences. Instead, all data will continue to be rendered with the aforementioned Nuxt Content Module. Most data should live in YAML files as structured blocks of strings. There may be call for some numerical data from time to time. We will not allow or support markdown or other rich text strings, with the notable exception that some text elements may need inline links, which we can store as single p tags with anchors in them, and render with Vue.js v-html attributes.

By making this decision, we will be simplifying and streamlining our architecture, and making changes to the website more predictable and manageable. The cost is that some content editors may need to wait longer for engineers to implement their changes. Fortunately, in 2022, we only saw about 2-3 requests per month that required this kind of work across the Buyer Experience repository and www-gitlab-com repository.

If, in any particular quarter, we spend more than two weeks doing content updates implemented by engineering, we should revisit the need for a content management system.

Developer Experience

Where Developer Documentation Exists

The README.md at the repository root and our /docs directory.

How It Works with www-gitlab-com

See /docs/deployments.md.

Changelog File and Semantic Versioning

The CHANGELOG.md is a file that contains the information of the changes made to the repository on every new version release. In our case, we are releasing a new version per iteration running a job at the end of it.

Every new release will create a tag in the repository's tag list and it is named using Semantic Versioning.

Why is it useful?

There are two main advantages of having this process in the project.

The first one is having versioning tags, sometimes it is necessary to make a rollback to a

previous version

due to a critical bug or an unexpected error. Each of these tags has a pipeline that can be run to deploy any version needed.

The second advantage is that having a Changelog file, there is a single source of truth and a history of every MAJOR, MINOR and PATCH version, making it easier to find any change without having to go back and search the project's board of past iterations.

How it works?

In order to accomplish the changelog automated updates, tag creation and version management, we are using semantic-release, which is a package that helps automate the whole process and has great community of developers behind it. The specific configuration and technical details can be found in the docs.

A manual/scheduled pipeline job will be run at the end of each iteration cycle, in order for semantic-release to detect correctly how to increase the release version, it needs Merge commits to follow the guidelines from conventional commits.

Here is the list of types that can be used in the commit message:

- build: Changes that affect the build system
- chore: Changes to external dependencies (gulp, npm)
- ci: Changes to our CI configuration files and scripts (gitlab-ci)
- docs: Documentation only changes
- feat: A new feature
- fix: A bug fix
- perf: A code change that improves performance
- refactor: A code change that neither fixes a bug nor adds a feature
- style: Changes that do not affect the meaning of the code (white-space, formatting, missing semi-colons, etc)
- test: Adding missing tests or correcting existing tests

Here are some commit message examples and how they affect the version increase:

Message	Version increase (v1.0.0)
fix(Hompage): Fix broken link	v1.0.1
feat(Hompage): Add new section	v1.1.0
feat(Services)!: Replace services module	v2.0.0
style(Hompage): Change button colors	No version increase

After running the job, a new tag is created, the changelog file is updated and the package.json version is increased.

Localization

Content Management

We implemented a localization solution for Buyer Experience using the Nuxt i18n module. This module provides a user-friendly platform for contributors to translate pages into their preferred language.

With the Nuxt i18n module, we can easily manage translations, create language-specific pages, and ensure a smooth and seamless user experience for our contributors. Anyone can now contribute to the localization efforts, making the Buyer Experience more accessible to a wider audience and improving the overall user experience.

Workflow

When a first-time user visits any page on the site, the system reads the preferred language from their browser. If their language preference matches one of the available languages on the site, the page will load in that language. Users will also be able to see a Language Selector on the site to change their preferred language if necessary.

If the user's preferred language cannot be detected, the page will load in the default language, and the Language Selector will still be visible, allowing users to switch to their preferred language. This ensures that all users can access the site's content in their desired language, regardless of whether their browser settings are supported or not.

Some pages on the site may not be available in all languages. In these cases, if a user navigates to a non-localized page, the page will appear in the default language.

To ensure a seamless user experience, site redirections are managed through a combination of Nuxt i18n module routing configuration and a custom routing configuration in the Digital Experience Navigation Repository. This dynamic redirection system automatically directs users to their preferred language page if it exists, or to the default language page if a localized version is not available.

We are committed to improving the site's localization efforts and providing users with the best possible experience on our site.

Learn More

For more technical details, please check [Language selector initial implementation](#)

SEO

No-index pages

In the common/no-index URLs file, we have a list of our no-index URLs to prevent hreflang from appearing on these pages. Please do not remove any pages from this file unless you're changing them to indexable URLs.

SECTION: marketing/digital-experience/engineering/code-review.md

Overview

Code reviews are mandatory for every merge request, you should get familiar with and follow our Code Review Guidelines outlined here.

These guidelines also describe who would need to review, approve and merge your, or a community member's, merge request. They also describe the review response time SLO's team members have to abide by.

Values

Every reviewer at GitLab must strive for our reviewer values.

Getting your merge request reviewed, approved, and merged

Acceptance Checklist

This checklist encourages the authors, reviewers, and maintainers of merge requests (MRs) to confirm changes were analyzed for high-impact risks to quality, performance, reliability, security, observability, and maintainability.

Using checklists improves quality in software engineering. This checklist is a straightforward tool to support and bolster the skills of contributors to the GitLab Marketing codebases.

Checklist for Submitting a Merge Request

1. Complete the Merge Request Template

Prefix the merge request with Draft: while you are building it.

Ensure all fields in the merge request template are properly filled out.

Provide a Comprehensive Description

Clearly describe what has changed, why the changes were necessary, and how they address the problem or feature request.

If applicable, mention any refactoring or architectural decisions made.

2. Link Related Issues

Reference any related issues, tickets, or user stories using appropriate syntax (e.g., related 1234).

Ensure the issues being referenced are correctly tagged and updated.

3. Document Testing Instructions

Provide clear, step-by-step instructions on how to test the changes locally or in a testing environment.

Mention any special configurations, test data, or preconditions required for testing.

If automated tests are included, specify their location and how to run them.

4. Deployment Steps

Detail the deployment process, including any scripts, commands, or configurations needed.

Mention any dependencies or services that need to be updated or restarted.

5. QA and Verification

Outline the QA process, specifying which environments (e.g., local, review app, production) will be used for validation.

List key scenarios and edge cases to be tested, including expected results.

6. Post-Deployment Validation

Provide a plan for validating the changes in the production environment post-deployment.

Include any monitoring or logging to watch for potential issues.

Detail a rollback plan in case of issues, specifying steps to revert the changes safely.

7. Request Feedback and Review

Remove Draft: from merge request title or explain why it is left in draft

Tag relevant team members for code review, QA, and any other necessary approvals.

Address any comments or requested changes promptly to ensure a smooth review process. The assignee is responsible for creating and tracking any follow-up issues to ensure they are completed in a timely manner.

Reviewer

All GitLab engineers can, and are encouraged to, perform a code review on merge requests of colleagues and community contributors. If you want to review merge requests, you can wait until someone assigns you one, but you are also more than welcome to browse the list of open merge requests and leave any feedback or questions you may have. Note that while all engineers can review all merge requests, the ability to accept merge requests is restricted to maintainers.

You can find someone to review your merge requests by:

1. Looking on the list of GitLab Digital Experience Team Members.
2. Utilizing Reviewer Roulette which will drop suggestion(s) for both a maintainer and reviewer currently available on our team. The suggestion will be listed as on comment on the change associated with your MR. Reviewer Roulette is only available for the Buyer Experience project
3. Utilizing the Gitlab Review Workload Dashboard this will randomly select a maintainer and reviewer for you. Gitlab Review Workload Dashboard is only available for the Buyer Experience project

Review turnaround time

Because unblocking others is always a top priority, reviewers are expected to review merge requests in a timely manner, even when this may negatively impact their other tasks and priorities.

Doing so allows everyone involved in the merge request to iterate faster as the context is fresh in memory, and improves contributors' experience significantly.

Review-response SLO

To ensure swift feedback to ready-to-review code, we maintain a Review-response Service-level Objective (SLO).

The SLO applies to GitLab team members, but not to other wider community contributors.

The SLO is defined as:

> Review-response SLO = (time when review is provided) - (time MR is assigned to reviewer)

The SLO value from Digital Experience GitLab team members is 2 business days

If you don't think you can review a merge request in the Review-response SLO time frame, let the author know as soon as possible in the comments (no later than 36 hours after first receiving the review request)

and try to help them find another reviewer or maintainer who is able to, so that they can be unblocked and get on with their work quickly. Remove yourself as a reviewer.

Of course, if you are out of office and have communicated this through your GitLab.com Status, authors are expected to realize this and find a different reviewer themselves.

When a merge request author has been blocked for longer than the Review-response SLO, they are free to remind the reviewer through Slack or add another reviewer.

Managing expectation

When you are assigned to review an MR and you are not able to get to it within the Review-response SLO, you should leave a comment on the MR informing the author of your delayed response. If possible, you should also indicate when the author can expect your feedback or help them find an alternative reviewer.

As the author of an MR you should reassign to another reviewer or maintainer if the Review-response SLO has not been met and you have been unable to contact the assignee.

Code Owner approvals

Some GitLab projects use GitLab's CODEOWNERS file feature to manage approvals for specific file paths and types. In the digital-experience/buyer-experience project, we use a combination of CODEOWNERS approval rules plus MR approval settings in order to follow segregation of duties best practices. This section describes the process for updating the eligible approvers for CODEOWNERS changes for the digital-experience/buyer-experience project.

The Code Owners for the CODEOWNERS file itself are managed with a rule in the file.

SECTION: marketing/digital-experience/engineering/code-standards.md

Overview

This document outlines the coding standards employed by the Digital Experience team across various projects and technology stacks. These guidelines ensure that our codebases remain clean, maintainable, and scalable as our ecosystem evolves. Key areas covered include project structure, naming conventions, component development, styling, JavaScript best practices, testing, and code review guidelines.

The goal is to foster a consistent development approach that enhances collaboration, reduces bugs, and delivers a high-quality user experience. By adhering to these standards, we strive to create a robust and efficient website aligned with modern web development best practices.

Project Structure

Define a clear and organized folder structure for all projects.
Keep related files and components together to improve discoverability and maintainability.

Naming Conventions

Use clear, descriptive, and consistent naming for files, components, and variables.

Component Development

Break down complex UI elements into smaller, reusable components.
Ensure components are self-contained and do not have side effects.

Linting

Use ESLint with a shared configuration to enforce code quality and consistency.

Accessibility

Follow WCAG guidelines to ensure the website is accessible to all users.
Regularly test components and pages with screen readers and other accessibility tools.

Testing

Ensure end-to-end tests cover critical user flows to prevent regressions.

Code Review Guidelines

Conduct thorough code reviews to maintain high code quality.
Focus on functionality, readability, and adherence to general code standards(SOLID, DRY, KISS)

Additional Resources

For more detailed guidelines and examples, refer to the Digital Experience repository you are working with, specifically the /docs directory.

By following these coding standards, we aim to maintain code quality, consistency, and readability across all projects.

You can also find a lot of useful insights in the product team's code review values·definitely worth a look!

SECTION: marketing/digital-experience/engineering/cwv.md

Core Web Vitals (CWV) are a set of performance metrics introduced by Google to measure key aspects of user experience on the web. These metrics focus on loading speed, interactivity, and visual stability, ensuring a smoother and more user-friendly experience. Understanding and optimizing Core Web Vitals is crucial for improving search rankings and enhancing user satisfaction.

Key Metrics

1. Largest Contentful Paint (LCP): Measures loading performance. LCP represents the time it takes for the largest visible content element (e.g., image or block of text) to appear in the viewport.

Target: LCP should occur within 2.5 seconds of when the page first starts loading.

2. Cumulative Layout Shift (CLS): Measures visual stability. CLS tracks how much the layout shifts during the loading process. A page with minimal unexpected shifts provides a better experience.

Target: A CLS score of less than 0.1.

3. Interaction to Next Paint (INP): Measures interactivity. INP evaluates how quickly a webpage responds to user inputs, such as clicks or keyboard events.

Target: INP should be less than 200 milliseconds.

Importance of Core Web Vitals

Core Web Vitals are part of Google's overall page experience signals, which assess how users perceive their experience interacting with a web page. Websites that score well on Core Web Vitals tend to perform better in search rankings and offer improved user engagement. Ensuring these metrics are optimized can lead to reduced bounce rates, higher conversions, and better SEO performance.

Tools for Monitoring Core Web Vitals

1. Google Analytics

Google Analytics is primarily used for tracking website visitors and understanding their behavior. While it doesn't focus directly on Core Web Vitals, the tool is useful for tracking user en